

RECURSO ORIGINAL KLAS

Python: fundamentos practicos

Sintaxis, estructuras y ejercicios para empezar con criterio

Categoria	Informatica
Universidad	Universidad Politecnica de Madrid
Asignatura	Programacion
Origen legal	Contenido original creado para KLAS. No reproduce materiales de terceros.

Que incluye

- Mapa de conceptos: Pensar antes de escribir codigo, Variables y tipos, Condiciones y bucles.
- Checklist final para repasar sin releer todo el documento.
- Contenido original preparado para la beta de KLAS.

Como usarlo

- Lee primero los apartados completos sin subrayar.
- Despues convierte cada bloque en dos preguntas de examen.
- Usa el checklist final para comprobar dominio real.

Guía de estudio

1. Pensar antes de escribir código

Programar no es memorizar sintaxis, sino convertir una idea en pasos verificables. Antes de abrir el editor, escribe entrada, proceso y salida esperada. Si no puedes explicarlo en lenguaje natural, el código saldrá frágil.

Python ayuda porque su sintaxis es legible, pero la legibilidad no sustituye al diseño. Nombres claros, funciones pequeñas y pruebas manuales sencillas evitan la mayor parte de errores iniciales.

2. Variables y tipos

Una variable nombra un valor. Los tipos habituales al empezar son números, cadenas, booleanos, listas y diccionarios. El tipo importa porque determina que operaciones tienen sentido.

Ejemplo: `precio = 12.5`, `nombre = 'Ana'`, `activo = True`. El buen nombre de variable reduce comentarios innecesarios.

3. Condiciones y bucles

Una condición decide caminos. Un bucle repite una operación. La trampa habitual es mezclar demasiadas condiciones dentro de un bucle largo. Si un bloque crece mucho, extrae una función.

Ejercicio: recorre una lista de notas, calcula la media y devuelve también cuántas superan 5. Primero hazlo sin función, después encapsúlalo.

4. Funciones

Una función debe tener una responsabilidad clara. Recibe datos, realiza una operación y devuelve un resultado. Si una función imprime, guarda, calcula y valida a la vez, será difícil de probar.

Ejemplo conceptual: `calcular_media(notas)` debe devolver un número. Otra función podría encargarse de formatear el resultado para pantalla.

5. Errores comunes

`IndexError` suele indicar acceso fuera de rango. `KeyError` aparece al pedir una clave inexistente en un diccionario. `TypeError` avisa de una operación incompatible. Leer el error completo ahorra tiempo.

Depurar no es adivinar: reproduce el fallo, reduce el caso, imprime valores intermedios y cambia una cosa cada vez.

Checklist de repaso

1. Escribe entrada, proceso y salida antes del código.

2.	Usa nombres de variables que expliquen el dominio.
3.	Separa calculo, validacion y presentacion.
4.	Lee el traceback desde la ultima linea relevante.

Nota de uso

Este documento es un recurso educativo introductorio. Puede servir como punto de partida, pero no sustituye bibliografia oficial, clases, normativa vigente ni criterio docente.